

ARM PrimeCell Driver

INT v1.5 (PL001)
API Specification

Primecell driver API description

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	F
Primecell driver identifier	INT
Code Version	1.5
Primecell identifier(s)	PL001 (Micropack)

Contents

1	ABOUT THIS DOCUMENT	6
1.1	Change control	6
1.1.1	Change history	6
2	SUMMARY	7
3	FEATURES	7
4	DRIVER USAGE	8
4.1	Configuring the Interrupt Controller	8
4.1.1	apINT_VERSION	8
4.1.2	apCONFIG_CORE_SUPPORTS	8
4.1.3	apOS_CONFIG_INT_REENTRANT	8
4.1.4	apOS_CONFIG_INT_USE_PREDISPATCH_CODE	9
4.1.5	apOS_CONFIG_INT_USE_POSTDISPATCH_CODE	9
4.1.6	apOS_CONFIG_INT_ISR_ISA	9
4.1.7	apOS_INT_CONFIG_SOURCES_USED	9
4.1.8	apOS_CONFIG_VIC_AT_0xFFFFF000	9
5	DETAILS	10
6	RELEASED FILES	10
7	PUBLIC API	11
7.1	Type Definitions	11
7.1.1	apINT_eInterruptSense	11
7.1.2	apINT_eInterruptType	11
7.1.3	apINT_rDispatchHandler	11
7.1.4	apINT_sInitialData	11
7.2	External Functions	12
7.2.1	apINT_ControllerDisable	12
7.2.2	apINT_ControllerRestore	12
7.2.3	apINT_HandlerChain	12
7.2.4	apINT_HandlerClear	13
7.2.5	apINT_HandlerSet	13
7.2.6	apINT_IRQEnabledMaskPtrGet	14
7.2.7	apINT_IRQPostDispatchSet	14
7.2.8	apINT_IRQPreDispatchSet	14
7.2.9	apINT_Initialize	15
7.2.10	apINT_InterruptClear	15
7.2.11	apINT_InterruptDisable	15
7.2.12	apINT_InterruptEnable	16
7.2.13	apINT_PrioritySet	16

7.2.14	apINT_ProgrammedInterruptClear	16
7.2.15	apINT_ProgrammedInterruptSet	17
7.2.16	apINT_SenseSet	17
7.3	Macros	18
7.3.1	apOS_INT_TOTAL_SOURCES_USED	18

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A		First release of API
B	15/11/00	Revised document to match desired template format
C	16/03/01	Added 'Driver Usage' section
D	09/08/01	Modified Interrupt Configuration header details
E	29/08/01	Modified 'apINT_VERSION' options
F	20/11/01	Modified 'apOS_CONFIG_INT_USE_PREDISPATCH_CODE' and 'apOS_CONFIG_INT_USE_POSTDISPATCH_CODE' options

2 SUMMARY

The INT peripheral is the Micropack Interrupt controller, present on the AP Motherboard of the Integrator development platform. It is not a PrimeCell, but is released in object code form with PrimeCell drivers for testing on development platforms.

3 FEATURES

The main features supported by the Micropack INT driver are:

- ◆ Interrupt dispatcher written in ARM assembler code.
- ◆ Binding and unbinding interrupt handlers to interrupt lines.
- ◆ Chained interrupt handlers (as with the Logic Modules on the Integrator platform).
- ◆ Enabling and disabling interrupts.
- ◆ Interrupt priorities set and changed dynamically at run-time.

4 DRIVER USAGE

4.1 Configuring the Interrupt Controller

The interrupt controller can be configured through the constants in **apintcfg.h**, which is in directory **apos**.

Note that the format of this file is unusual, as it is included by both assembler and C files. The main difference is that the preprocessor instruction '# define' must be written with a space after the '#' character.

4.1.1 apINT_VERSION

This specifies the version of the interrupt controller and should be one of the options below:

4.1.1.1 apVERSION_INTEGRATOR

This assumes a RPS interrupt controller of the type present on an Integrator development platform.

If running on an Integrator platform this assumes a RPS controller in the Integrator AP, with an optional logic module. If present, the logic module also includes a RPS controller.

4.1.1.2 apVERSION_ARMULATOR

ARMulator model, assuming a single RPS controller.

4.1.1.3 apVERSION_VECTORED

A single PrimeCell vectored interrupt controller PL190

4.1.1.4 apVERSION_VIC_ON_LM

Integrator platform - this assumes a RPS controller in the Integrator AP, with an optional logic module. If present, the logic module includes a PrimeCell vectored PL190 controller.

4.1.1.5 apVERSION_NONE_ON_LM

Integrator platform - this assumes a RPS controller in the Integrator AP, with an optional logic module. If present, the logic module includes no interrupt controller.

4.1.2 apCONFIG_CORE_SUPPORTS

This should be defined as one of the following:

- ◆ apCONFIG_ARM_AND_THUMB indicates that the core can support both ARM and Thumb states and the assembler code is then compiled for interworking.
- ◆ apCONFIG_ARM_ONLY indicates that the core will only support ARM code (e.g. StrongARM).

4.1.3 apos_CONFIG_INT_REENTRANT

This exists for expansion only and must be defined as 0.

4.1.4 apOS_CONFIG_INT_USE_PREDISPATCH_CODE

- ◆ If defined as 1, the Interrupt controller will execute a specified routine before calling each Interrupt Service Routine.
- ◆ If defined as 0, no pre-dispatch routine is available, but there will be a slight speed gain.

This routine is specified using apINT_IRQPreDispatchSet() function.

4.1.5 apOS_CONFIG_INT_USE_POSTDISPATCH_CODE

- ◆ If defined as 1, the Interrupt controller will execute a specified routine after calling each Interrupt Service Routine.
- ◆ If defined as 0, no post-dispatch routine is available, but there will be a slight speed gain.

This routine is specified using apINT_IRQPostDispatchSet() function.

4.1.6 apOS_CONFIG_INT_ISR_ISA

- ◆ If defined as apCONFIG_ARM_ONLY, interrupt service routines are expected to be in ARM code. The linker will add veneers when interworking with Thumb ISRs as usual.
- ◆ If defined as apCONFIG_THUMB_ONLY, interrupt service routines are expected to be in Thumb code. The linker will add veneers when interworking with ARM ISRs as usual.

4.1.7 apOS_INT_CONFIG_SOURCES_USED

Each interrupt controller can control up to 32 interrupt sources. By default, enough space is assigned for this many sources. However, if a lesser number of sources are required, this may be specified by defining the constant apOS_INT_CONFIG_SOURCES_USED. This constant should specify the total number of interrupt sources used across all interrupt controllers as below

```
# define apOS_INT_CONFIG_SOURCES_USED 16
```

4.1.8 apOS_CONFIG_VIC_AT_0xFFFFF000

In a real system, a Vectored Interrupt Controller (if present) should be placed at 0xFFFFF000 in the memory map, although this may not be possible in a development system.

This will mean that a dispatcher need not be called for interrupts, so saving a number of cycles. See the PL190 documentation.

This is indicated by setting this constant as below:

```
# define apOS_CONFIG_VIC_AT_0xFFFFF000 1
```

5 DETAILS

◆ Peak performance obtained in tests:	N/A
◆ ROM size (RO code + RO data) (release build):	1800 Bytes
◆ RAM size (ZI data) per instance (release build):	500 Bytes
◆ Does this driver support multiple Primecells?:	NO
◆ Can these be detected at run-time?	-----
◆ Does this driver support both endian-nesses? ¹	YES
◆ Does this driver support Thumb? ²	YES

6 RELEASED FILES

PL001-ISPC-1000-	This API document
PL001-TREP-1000	Test Results document
apINT/apINT.h	Public header file
apINT/INT.h	Private header file
apINT/dispatch.s	Interrupt dispatcher
apINT/INT.c	Source code
apINT/INTtst.h	Self-test module

¹ To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

² The driver may include ARM code, but this must interwork with a Thumb build.

7 PUBLIC API

7.1 Type Definitions

7.1.1 apINT_eInterruptSense

Enumeration of the possible combinations of polarity and edge/level sensitivity of a peripheral's interrupt signal.

Declaration

```
typedef enum apINT_eInterruptSense
```

Elements

apINT_ACTIVE_HIGH	Active high (level sensitive)
apINT_ACTIVE_LOW	Active low (level sensitive)
apINT_BOTH_EDGES	Both edges (edge sensitive)
apINT_DEFAULT_SENSE	Leave as the default reset value.
apINT_FALLING_EDGE	Falling edge (edge sensitive)
apINT_RISING_EDGE	Rising edge (edge sensitive)

7.1.2 apINT_eInterruptType

Enumeration of the possible routes by which an interrupt generates a processor exception, either to IRQ or FIQ.

Declaration

```
typedef enum apINT_eInterruptType
```

Elements

apINT_FIQ	Generates an FIQ exception
apINT_IRQ	Generates an IRQ exception

7.1.3 apINT_rDispatchHandler

Function type for use with apINT_PreDispatchSet() and apINT_PostDispatchSet().

Declaration

```
typedef void (apINT_rDispatchHandler) (void)
```

Remarks

The pre and post dispatch handlers are intended for additional processing each time an IRQ interrupt occurs. Typically, an OS will save the interrupted process state, before calling the interrupt handler, and then restore it afterwards. It may also choose to swap out the interrupted process and schedule one of higher priority. Note: These are not APCS compliant functions, as they do not return to the link register, but must branch directly back into the interrupt dispatch code.

7.1.4 apINT_sInitialData

Structure for passing the default interrupt sense and priority into the interrupt initialization function

apINT_Initialize.

Declaration

```
struct apINT_sInitialData
{
    apINT_eInterruptSense eSense;
    UBYTE8                Priority;
}
```

Elements

Priority	The default interrupt priority, values are as defined for the function apINT_PrioritySet()
eSense	The default sense to assign to all interrupts

7.2 External Functions

7.2.1 apINT_ControllerDisable

Temporarily disables all interrupts for a particular interrupt controller.

Declaration

```
PUBLIC void apINT_ControllerDisable( apOS_INT_oId oId )
```

Parameters

old	The interrupt controller to disable.
-----	--------------------------------------

Remarks

Disables all of the interrupt sources for this controller in hardware (keeping a software copy for re-enabling them using apINT_ControllerRestore()). All subsequent accesses to the interrupt module act only on this software copy, until such time as the function apINT_ControllerRestore() is called to countermand this function. Any other interrupt controllers which funnel into one of the interrupt sources of the controller specified will be disabled by this call. Do not nest apINT_ControllerDisable()/apINT_ControllerRestore() pairs. If you were to (for example Disable,Disable,Restore,Restore) then the first call to apINT_ControllerRestore() will restore the interrupts (not the second as might be expected).

7.2.2 apINT_ControllerRestore

Use to return to normal operation after a call to apINT_ControllerDisable().

Declaration

```
PUBLIC void apINT_ControllerRestore( apOS_INT_oId oId )
```

Parameters

old	The interrupt controller to restore.
-----	--------------------------------------

Remarks

Re-enables all the interrupts which were enabled before a call to apINT_ControllerDisable(). Copies the software version of the interrupt sources which are enabled to the hardware, and allows the other interrupt functions to access the hardware directly again.

7.2.3 apINT_HandlerChain

Chains an interrupt controller from an interrupt source in a different interrupt controller.

Declaration

```
PUBLIC void apINT_HandlerChain( CONST apOS_INT_oInterruptSource oSource,
                               apOS_INT_oId                      oId )
```

Parameters

oSource	The interrupt source to attach the handler to.
oId	The interrupt controller to be chained to this source.

Remarks

Only one controller or handler can be assigned to each source, and if a second controller or handler is assigned to a source, then the first will be removed, and then the second added. This function does not return any errors.

The interrupt source that the controller is chained from must be subsequently enabled in order for interrupt requests from the chained controller to interrupt the processor.

The chaining can be broken by calling `apINT_HandlerClear()` for the interrupt source the controller is chained from.

7.2.4 apINT_HandlerClear

Removes an interrupt handling function for a specified interrupt source.

Declaration

```
PUBLIC void apINT_HandlerClear( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource	The interrupt source to remove the handler from.
----------------	--

Remarks

As only one handler can be assigned to each source, removing it will leave no handlers for a particular interrupt. Therefore, this routine also disables the interrupt by calling the function `apINT_InterruptDisable()`. Does not return any errors.

7.2.5 apINT_HandlerSet

Registers an interrupt handling function for a specified interrupt source. Also registers an ID number which will be passed to the interrupt handler when the interrupt occurs.

Declaration

```
PUBLIC void apINT_HandlerSet( CONST apOS_INT_oInterruptSource oSource,
                              apOS_INT_rServiceRoutine        rHandler,
                              UWWORD32                        Parameter )
```

Parameters

oSource	The interrupt source to attach the handler to.
rHandler	The interrupt handling function.
Parameter	Parameter to be associated with this interrupt source and passed to the handler function when the interrupt occurs.

Remarks

Only one handler can be assigned to each source, and if a second is assigned to a source, then the first will be removed, and then the second added. This function does not return any errors.

The handler will be assigned to either IRQ or FIQ depending on how the source was set up using the function

apINT_SelectPriority().

The ID number specified to be associated with the interrupt source is designed to allow the same interrupt handling function to be used for multiple interrupt sources (for example in a system where there are several identical timers). The ID number is passed to the interrupt handling function (as the first parameter, in R0) to allow it to know which of the interrupt sources has generated the interrupt.

Note: *Always register a handling function before enabling the corresponding interrupt source.*

7.2.6 apINT_IRQEnabledMaskPtrGet

This function returns a pointer to the software version of the register indicating which (IRQ) interrupts the user has asked to be enabled for the root interrupt controller (number 0).

Declaration

```
PUBLIC UWORD32 *apINT_IRQEnabledMaskPtrGet( void )
```

Remarks

This is provided for backdoor optimisations in critical pieces of assembler code. It is not recommended to use this function unless you (a) really have to, and (b) you really know how the rest of this code works. Don't use if compiled for nested/re-entrant interrupts. Don't alter the hardware register based on this if in between a apINT_ControllerDisable() and apINT_ControllerRestore() pair. Only alter this register when all interrupts are disabled.

Note: for some interrupt controllers this value is for all interrupts (IRQ and FIQ), whereas for interrupt controllers with separate IRQ and FIQ enable registers it is for IRQs only.

7.2.7 apINT_IRQPostDispatchSet

Registers a function to be called when an IRQ interrupt exception occurs, but after the interrupt dispatcher (and the individual interrupt handlers) are called.

Declaration

```
PUBLIC void apINT_IRQPostDispatchSet( apINT_rDispatchHandler rPostDispatchHandler )
```

Parameters

rPostDispatchHandler	Pointer to the function to be registered
----------------------	--

Remarks

Registering a function supercedes any previous registered function. If a NULL pointer is passed, the post-dispatcher function will not be called.

Note: *For this to have effect, the pre-processor constant apOS_CONFIG_INT_USE_POSTDISPATCH_CODE must have been #defined to 1.*

7.2.8 apINT_IRQPreDispatchSet

Registers a function to be called when an IRQ interrupt exception occurs, but before the interrupt dispatcher (and the individual interrupt handlers) are called.

Declaration

```
PUBLIC void apINT_IRQPreDispatchSet( apINT_rDispatchHandler rPreDispatchHandler )
```

Parameters

rPreDispatchHandler	Pointer to the function to be registered
---------------------	--

Remarks

Registering a function supercedes any previous registered function. If a NULL pointer is passed, the pre-dispatcher function will not be called.

Note: *For this to have effect, the pre-processor constant `apOS_CONFIG_INT_USE_PREDISPATCH_CODE` must have been #defined to 1.*

7.2.9 apINT_Initialize

Initialises the interrupt system, configures all the interrupt sources to the same specified default, and specifies the base address of the interrupt hardware registers.

Declaration

```
PUBLIC apError apINT_Initialize( apOS_INT_oId          oId,  
                               apOS_System_eBaseAddress eBase,  
                               UWORD32                Interrupts,  
                               CONST apOS_INT_oInterruptSource *pSources,  
                               apINT_sInitialData      *pInitial )
```

Parameters

old	The interrupt controller to initialise
eBase	The base address of the interrupt hardware registers.
Interrupts	Pass as aNULL
pSources	Pass as aNULL
pInitial	Pointer to structure containing default sense and priorities to be used for each interrupt source.

Remarks

The specified base address is remembered for all future accesses to the interrupt hardware using this API. All interrupt sources are disabled, by masking out the interrupts. All interrupt handlers are removed. All interrupts are set to the specified default sense and type. All interrupts are assigned to IRQ by default where physically possible, and with the default priority specified.

7.2.10 apINT_InterruptClear

Clears the specified interrupt source.

Declaration

```
PUBLIC void apINT_InterruptClear( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource	The interrupt source to clear
---------	-------------------------------

Remarks

Writes to the interrupt controller clear register to clear the specified interrupt. This function does not return any errors.

Note, before clearing the interrupt in the interrupt controller, the peripheral generating the interrupt should be serviced to prevent the interrupt being asserted multiple times for the same reason.

7.2.11 apINT_InterruptDisable

Disables the specified interrupt source from producing an interrupt.

Declaration

```
PUBLIC void apINT_InterruptDisable( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource The interrupt source to disable

Remarks

Writes to the interrupt controller mask to disable the specified interrupt. All other interrupt sources remain unaffected. This function does not return any errors.

7.2.12 apINT_InterruptEnable

Enables the specified interrupt source.

Declaration

```
PUBLIC apError apINT_InterruptEnable( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource The interrupt source to enable.

Remarks

Writes to the interrupt controller mask to enable the interrupt. Whether the source generates either an IRQ or FIQ should have been prechosen by the user by calling apINT_PrioritySet() (otherwise the default action is to assign it to IRQ).

Always ensure that you have registered an interrupt handler for a particular source before enabling its interrupt. Otherwise the system will try to detect that this situation has occurred and ignore the request to enable the interrupt. This function does not return any errors.

7.2.13 apINT_PrioritySet

Selects whether the specified interrupt source would either produce an IRQ or FIQ interrupt exception. Also records the priority level of the interrupt, this is used to determine which interrupts get serviced first if there are multiple outstanding active interrupt sources.

Declaration

```
PUBLIC void apINT_PrioritySet( CONST apOS_INT_oInterruptSource oSource,
                             apINT_eInterruptType           eType,
                             UBYTE8                          Priority )
```

Parameters

oSource The interrupt source to enable

eType Which exception to produce, either apINT_FIQ or apINT_IRQ.

Priority The interrupt priority, integer from 0 (lowest priority) to 31 (highest priority).

Remarks

Selects the interrupt type for this interrupt source, the default (if this function is not called) is that the interrupt is assumed to be an IRQ, with default priority (specified when apINT_Initialize() was called)

Note, this function does not enable the interrupt. To enable the interrupt call the function apINT_InterruptEnable().

Note, this causes an update of the interrupt dispatch table during which all interrupts will be temporarily disabled.

7.2.14 apINT_ProgrammedInterruptClear

Clears a previously set user specified programmable interrupt.

Declaration

```
PUBLIC apError apINT_ProgrammedInterruptClear(  
                                            CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource The interrupt source to clear.

7.2.15 apINT_ProgrammedInterruptSet

Generates a user specified programmable interrupt. This will interrupt the processor, if the interrupt selected has been enabled.

Declaration

```
PUBLIC apError apINT_ProgrammedInterruptSet(  
                                            CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource The interrupt source to set.

Remarks

Will only work if the interrupt hardware supports programmable interrupts. Some hardware only partially supports programmable interrupts, by providing just one independent programmable interrupt source, rather than allowing any interrupt source to be programmed.

7.2.16 apINT_SenseSet

Selects the interrupt type. The type specifies whether the interrupt is level low, level high, rising edge, falling edge or sensitive to both edges.

Declaration

```
PUBLIC apError apINT_SenseSet( CONST apOS_INT_oInterruptSource oSource,  
                             apINT_eInterruptSense           eSense )
```

Parameters

oSource The interrupt source sensitivity to change
eSense The polarity and edge/level sensitivity for this source.

Return Value

apERR_NONE	Interrupt source sensitivity set, no errors
apERR_UNSUPPORTED	The sensitivity type is not supported by this hardware

Remarks

Selects the interrupt sensitivity for this interrupt source, the default (if this function is not called) is determined by the settings specified by the call to apINT_Initialize().

All interrupts may be temporarily disabled, and any outstanding latched edge-sensitive interrupts are cleared. Any interrupts which were already enabled, are re-enabled.

To enable an interrupt which was previously disabled, call the function apINT_InterruptEnable().

7.3 Macros

7.3.1 apOS_INT_TOTAL_SOURCES_USED

Although each interrupt controller may control 32 interrupt sources, you may optionally define apOS_INT_TOTAL_SOURCES_USED to specify the total number of sources in use across all interrupt controllers. This will reduce the storage space used by the interrupt controller.

Declaration

```
#define apOS_INT_TOTAL_SOURCES_USED 16
```